CAPÍTULO 6. GITHUB Y OTROS SERVICIOS DE ALOJAMIENTO DE REPOSITORIOS

Introducción

Los servicios de alojamiento de repositorios permiten almacenar proyectos de Git en plataformas en línea y facilitan el trabajo colaborativo entre diferentes usuarios. Estas plataformas ofrecen herramientas para administrar archivos, revisar cambios, gestionar errores y coordinar el desarrollo de proyectos.

Entre los servicios más conocidos se encuentra GitHub, que permite alojar repositorios públicos y privados, colaborar mediante solicitudes de incorporación de cambios y documentar proyectos mediante archivos como README.md.

En este capítulo se explican las principales características de GitHub, el procedimiento para publicar un repositorio, la colaboración mediante bifurcaciones y solicitudes de incorporación de cambios, y algunas herramientas adicionales para administrar proyectos.

1. ¿Qué es GitHub?

GitHub es una plataforma de alojamiento y colaboración que utiliza Git para administrar las versiones de los proyectos. Aunque Git y GitHub están relacionados, no son lo mismo.

Git es el sistema de control de versiones que permite registrar modificaciones, crear ramas y consultar el historial de un proyecto. GitHub, por su parte, es un servicio en línea que permite almacenar repositorios Git y facilita la colaboración entre sus usuarios.

Entre sus principales funcionalidades se encuentran:

- Almacenar repositorios públicos y privados.
- Consultar archivos e historial de cambios.
- Gestionar ramas y solicitudes de incorporación de cambios.
- Registrar y hacer seguimiento de errores mediante *issues*.
- Documentar proyectos mediante archivos como README.md.
- Administrar permisos de acceso para los colaboradores.
- Automatizar tareas mediante herramientas de integración y despliegue continuo.

Estas características hacen de GitHub una herramienta útil tanto para proyectos académicos como para proyectos profesionales.

2. Crear una cuenta en GitHub

Para utilizar las funciones de colaboración de GitHub, es necesario contar con una cuenta en la plataforma.

El procedimiento general es el siguiente:

1. Ingresar al sitio oficial: https://github.com/.
2. Seleccionar la opción para registrarse.
3. Introducir la información solicitada y completar el proceso de verificación.
4. Iniciar sesión con la cuenta creada.
5. Acceder al panel principal de GitHub.

Una vez iniciada la sesión, se pueden crear repositorios, consultar proyectos y participar en trabajos colaborativos.

3. Crear un repositorio en GitHub

Un repositorio es el espacio donde se almacenan los archivos de un proyecto y su historial de versiones.

Para crear uno en GitHub, se deben seguir estos pasos:

1. Iniciar sesión en la plataforma.
2. Seleccionar la opción para crear un nuevo repositorio.
3. Asignar un nombre descriptivo, por ejemplo, tutorial-git.
4. Escribir una descripción breve del proyecto, si se considera necesario.
5. Elegir la visibilidad del repositorio: público o privado.
6. Configurar las opciones iniciales, como la creación de un archivo README.md, si corresponde.
7. Confirmar la creación del repositorio.

Al finalizar, GitHub mostrará la página principal del repositorio, desde la cual se podrán consultar los archivos, las ramas y las confirmaciones.

Si el proyecto ya existe en el computador, es importante revisar si el repositorio remoto está vacío o si ya contiene archivos e historial antes de realizar la conexión.

4. Publicar un repositorio local en GitHub

Cuando un proyecto ya está creado en el computador mediante Git, puede conectarse con un repositorio de GitHub para compartir su contenido.

Primero, se debe verificar el estado del repositorio local:

```
git status
```

Después, se revisa si ya existe un repositorio remoto configurado:

```
git remote -v
```

Si no existe un remoto y el repositorio de GitHub está preparado para recibir el proyecto, se puede agregar su dirección:

```
git remote add origin https://github.com/usuario/tutorial-git.git
```

La dirección es un ejemplo y debe sustituirse por la dirección real del repositorio creado en GitHub.

A continuación, se comprueba el nombre de la rama actual:

```
git branch
```

Si la rama principal se llama main, se pueden enviar los cambios mediante:

```
git push -u origin main
```

La opción -u establece una relación de seguimiento entre la rama local y la remota.

Este procedimiento requiere que los cambios estén registrados mediante confirmaciones, que la rama esté correctamente configurada y que el usuario tenga los permisos necesarios para publicar el contenido.

Si el repositorio de GitHub ya contiene confirmaciones que no están presentes en el repositorio local, puede ser necesario sincronizar los historiales antes de enviar los cambios.

5. El archivo README.md

El archivo README.md es un documento de presentación que permite explicar el propósito de un proyecto y orientar a quienes visitan el repositorio. Generalmente, está escrito en Markdown y GitHub lo muestra en la página principal del repositorio.

Un buen archivo README.md debe incluir información como:

- Nombre del proyecto.
- Descripción y objetivo.
- Contenido del repositorio.
- Instrucciones básicas de uso, cuando sean necesarias.
- Herramientas utilizadas.
- Autor o autores del proyecto.

- Fuentes de consulta.

Para crear un archivo desde la terminal, se puede utilizar un editor de texto o un comando como:

```
echo "# Tutorial de Git" > README.md
```

Este comando crea el archivo o reemplaza su contenido existente. Por eso, si ya existe un README.md con información importante, es preferible editarlo sin sobrescribirlo.

Después de modificarlo, se pueden registrar los cambios:

```
git add README.md

git commit -m "Agrega documentación del proyecto"

git push
```

El comando git push funciona si la rama actual ya tiene configurado su repositorio remoto de seguimiento. De lo contrario, se debe especificar el remoto y la rama correspondientes.

6. Markdown para documentar proyectos

Markdown es un lenguaje de marcado ligero que permite organizar documentos mediante títulos, listas, enlaces, imágenes y otros elementos de formato. Se utiliza frecuentemente en GitHub para redactar archivos README.md y otros documentos de apoyo.

Algunos ejemplos de su sintaxis son los siguientes:

Títulos:

```
# Título principal

## Segundo nivel

### Tercer nivel
```

Listas:

```
- Primer elemento

- Segundo elemento

- Tercer elemento
```

Texto en negrita:

```
**Texto importante**
```

Enlaces:

[Documentación oficial de Git](https://git-scm.com/doc)

Bloques de código:

```bash
git status
git add .
git commit -m "Registra cambios"
```

Esta sintaxis permite presentar la información de manera clara y ordenada, especialmente cuando se documentan instrucciones técnicas.

7. **Issues: seguimiento de tareas y errores**

Los *issues* son herramientas de GitHub que permiten registrar problemas, preguntas, propuestas de mejora y tareas relacionadas con un proyecto.

Por ejemplo, en un proyecto académico se puede crear un *issue* para informar que falta un capítulo, que un comando no funciona o que es necesario revisar la documentación.

Para crear un *issue*, generalmente se deben seguir estos pasos:

1. Ingresar al repositorio correspondiente.
2. Abrir la sección denominada **Issues**, si está habilitada.
3. Seleccionar la opción para crear un nuevo *issue*.
4. Escribir un título claro y una descripción del problema o tarea.
5. Agregar información relevante y crear el registro.

Los *issues* facilitan la organización del trabajo porque permiten identificar pendientes, asignar responsables y mantener un registro de las actividades que necesitan atención.

La disponibilidad de esta herramienta depende de la configuración y los permisos del repositorio.

8. **Pull requests y revisión de cambios**

Las solicitudes de incorporación de cambios, conocidas como *pull requests*, permiten proponer modificaciones a una rama de un repositorio para que sean revisadas antes de integrarlas.

Este mecanismo se utiliza especialmente cuando los colaboradores trabajan en ramas independientes o cuando una persona propone cambios a un proyecto del que no es administradora.

El proceso general consiste en crear una rama, modificar los archivos, registrar los cambios y enviarlos a GitHub. Después, se abre una solicitud de incorporación de cambios y se describen las modificaciones propuestas.

Los responsables del proyecto pueden revisar las diferencias entre las ramas, realizar comentarios, solicitar ajustes y aprobar la integración.

Las *pull requests* ayudan a detectar errores y facilitan la comunicación entre los participantes antes de incorporar modificaciones a la rama principal.

9. Bifurcar un repositorio (*fork*)

Una bifurcación o *fork* es una copia de un repositorio que se crea dentro de otra cuenta o espacio de GitHub. Permite trabajar de forma independiente sobre un proyecto sin modificar directamente el repositorio original.

Este procedimiento es habitual en proyectos de código abierto, donde diferentes personas pueden proponer mejoras aunque no tengan permisos de escritura sobre el proyecto principal.

El proceso general consiste en:

1. Abrir el repositorio que se desea utilizar.
2. Seleccionar la opción **Fork**, cuando esté disponible.
3. Elegir la cuenta o el espacio donde se creará la bifurcación.
4. Clonar la bifurcación en el computador.
5. Crear una rama y realizar los cambios.
6. Enviar las modificaciones a la bifurcación personal.
7. Proponer los cambios al proyecto original mediante una *pull request*, si corresponde.

Una bifurcación en GitHub no debe confundirse con una rama de Git. La rama es una línea de desarrollo dentro de un repositorio, mientras que el *fork* crea una copia del repositorio en la plataforma.

10. Otros servicios de alojamiento de repositorios

Aunque GitHub es una de las plataformas más conocidas, existen otros servicios que permiten alojar repositorios Git y facilitar el trabajo colaborativo.

Entre ellos se encuentran:

- **GitLab:** ofrece alojamiento de repositorios, revisión de cambios, gestión de tareas y herramientas de integración y despliegue continuo.
- **Bitbucket:** permite alojar repositorios Git y colaborar mediante herramientas de revisión de código y administración de proyectos.
- **Servidores Git propios:** algunas organizaciones alojan sus repositorios en infraestructura administrada internamente para controlar el acceso y la seguridad.

La elección de una plataforma depende de factores como las necesidades del proyecto, los permisos requeridos, las herramientas disponibles y las políticas de la organización.

11. Ejercicio práctico

El siguiente ejercicio permite aplicar los conocimientos relacionados con GitHub y la publicación de proyectos.

Paso 1. Ingresar a GitHub y abrir el repositorio creado para el tutorial.

Paso 2. Verificar que el repositorio contenga los documentos de los capítulos elaborados.

Paso 3. Comprobar que exista un archivo README.md con una descripción del proyecto y de su contenido.

Paso 4. Si el archivo README.md necesita modificaciones, editarlo para explicar el objetivo del tutorial y enumerar los documentos disponibles.

Paso 5. Si se realizaron cambios desde el computador, comprobar el estado del repositorio:

```
git status
```

Paso 6. Agregar los archivos modificados al área de preparación. Por ejemplo:

```
git add README.md
```

Si también se modificaron otros documentos, se deben agregar de acuerdo con los cambios realizados.

Paso 7. Registrar las modificaciones:

```
git commit -m "Actualiza documentación del tutorial"
```

Paso 8. Enviar los cambios a GitHub:

```
git push
```

Este último comando requiere que la rama tenga configurado el seguimiento remoto. Si no lo tiene, se debe especificar el remoto y la rama.

Paso 9. Abrir el repositorio en el navegador y comprobar que los cambios se encuentren publicados.

Paso 10. Copiar la dirección del repositorio para utilizarla como evidencia de entrega.

Al finalizar, el repositorio deberá mostrar los documentos elaborados y una descripción clara del proyecto.

Conclusión

GitHub y los demás servicios de alojamiento de repositorios amplían las posibilidades de Git al facilitar el almacenamiento, la documentación y la colaboración en proyectos. Sus herramientas permiten administrar archivos, registrar tareas, revisar modificaciones y proponer cambios de forma organizada.

El uso de archivos README.md, *issues*, ramas, bifurcaciones y solicitudes de incorporación de cambios contribuye a mejorar la comunicación y la trazabilidad del trabajo. Para un proyecto académico, estas herramientas también permiten presentar de manera ordenada los documentos elaborados y facilitar su revisión por parte del docente.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Disponible en: https://git-scm.com/book/es/v2

Documentación oficial de GitHub. *GitHub Docs*. Disponible en: https://docs.github.com/es